

Beancat's Python Lecture

Lecture 1

本讲内容

 表达式与变量

 函数定义

 条件控制

 高阶函数

最后更新：2026 年 7 月 18 日

1 Python 入门

Python 是一种广泛使用的**解释型**、高级、通用编程语言。

解释型的意思是：Python 会将代码一句一句直接执行，不需要像**编译型**语言（如 C 语言）那样，先经过编译器编译为机器码再运行。

1.1 在终端中运行 Python

💡 小贴士

1. 按下 `Cmd + Space`，搜索 **Terminal**（终端），打开它。
2. 输入 `python3` 并回车，即可启动 Python 解释器。

在终端中运行 Python 可以一句一句地交互式执行代码，非常适合初学者学习语法。

2 基本表达式

Python 会自动计算每个表达式的值并返回。试着在解释器中输入以下内容：

```
1 2
2 2
3 -2
4 -2
```

2.1 算术运算

```
1 1 + 2
2 3
3 3 - 2
4 1
5 2 * 2
6 4
7 3 / 2
8 1.5
9 3 // 2
10 1
11 5 % 3
12 2
```

2.2 布尔表达式

```
1 3 == 2
2 False
3 2 < 3
4 True
5 3 != 2
6 True
7 5 >= 5
8 True
```

3 变量与值绑定

Python 中的变量将名称与值绑定在一起。等号(=)是**赋值**运算符，而不是数学意义上的等式。

```
1 x = 3
2 x
3 3
4 2 * x
5 6
6 x = x + 2
7 x
8 5
```

⚠ 注意

在 Python 中，`x = x + 2` 的意思是：取出 `x` 当前的值，加上 2，再将名称 `x` 绑定到新结果。这**不是**代数方程，而是一系列计算步骤。

4 函数

函数是对输入到输出的映射： $f(x)$ 。Python 中有很多内置函数（如 `max`、`min`、`abs`），同时你也可以自定义函数。

4.1 定义简单函数

```
1 def add_one(x):
2     ...     return x + 1
3     ...
4 add_one(3)
5 4
6 add_one(5)
7 6
8 add_one(add_one(3))
9 5
```

```
1 def square(x):
2     ...     return x * x
3     ...
4 square(3)
5 9
6 square(add_one(3))
7 16
```

4.2 多参数函数

```
1 def x_plus_y(x, y):
2     ...     return x + y
3     ...
4 x_plus_y(1, 2)
5 3
```

4.3 函数也是值

Python 中的函数是**一等公民**——你可以像绑定数字一样，将函数绑定到变量名：

```
1 s = square
2 s(2)
3 4
```

这一点至关重要，后面学习高阶函数时会大有用处。

5 条件控制语句

5.1 print 函数

在深入条件语句之前，先来理解 `print`。`print` 的功能是在屏幕上打印内容，但它的返回值是 `None`。这两者需要区分开来：

```
1 3
2 3
3 print(3)
4 3
```

3 直接输入会返回 3（由解释器显示出来），而 `print(3)` 虽然也显示了 3，但那是 `print` 的副作用；`print` 本身的返回值是 `None`：

```
1 print(print(3))
2 3
3 None
4
```

`print` 在测试和调试代码时非常有用。

5.2 if 语句

`if` 会判断后面表达式的值。如果表达式的值为 `True`，则执行下方缩进的代码块。如果有 `else` 子句，则当条件为 `False` 时执行 `else` 下方的代码块。

```
1 def python(x):
2     ...     if x > 0:
3     ...         print("positive")
4     ...     else:
5     ...         print("not positive")
6     ...
7 python(3)
8 positive
9 python(-3)
10 not positive
```

```
11 python(0)
12 not positive
```

5.3 elif 子句

elif（即“else if”的缩写）可以串联多个条件。Python 会按顺序依次检查每个条件，第一个为 True 的条件对应的代码块会被执行，其余则被跳过。如果所有条件都不满足，则执行 else（如果有的话）。

```
1 def abs(x):
2     ...     if x > 0:
3     ...         return x
4     ...     elif x == 0:
5     ...         return x
6     ...     else:
7     ...         return -x
8     ...
9 abs(3)
10 3
11 abs(-3)
12 3
```

💡 小贴士

上面 `elif x == 0` 的分支**并不是必需的**——这里只是为了演示 `elif` 的写法。实际使用中，前两种情况可以直接合并为 `if x >= 0`。

6 高阶函数（HOF）

这是本节课最具挑战性的部分。

高阶函数（Higher-Order Function，简称 HOF）是指接收另一个函数作为参数和/或返回一个函数作为结果的函数。函数和数字一样，是可以被传递和操作的值。

6.1 函数作为参数

```
1 def square(x):
2     ...     return x * x
3     ...
4 def cube(x):
5     ...     return x * x * x
```

```
6 ...
7 square(2)
8 4
9 cube(2)
10 8
```

将一个函数作为参数传递给另一个函数：

```
1 def change(f):
2     ...     return f
3 ...
4 a = change(square)
5 a(2)
6 4
7 a = change(cube)
8 a(3)
9 27
```

```
1 def change(x, f):
2     ...     return f(x)
3 ...
4 change(3, cube)
5 27
```

关键理解：change（第二个版本）并不关心 `f` 具体做了什么——它只是用 `x` 去调用 `f`，然后返回结果。这种抽象让我们可以编写通用的、可复用的代码。

6.2 函数作为返回值

高阶函数真正的威力在于：一个函数可以返回另一个函数。

```
1 def make_adder(n):
2     ...     def adder(x):
3     ...         return x + n
4     ...     return adder
5 ...
6 add_3 = make_adder(3)
7 add_3(3)
8 6
```

```
9   add_3(6)
10  9
11   add_5 = make_adder(5)
12   add_5(5)
13  10
```

⚠ 注意

这是怎么做到的？内部函数 `adder` 会“记住”它被创建时 `n` 的值。这种机制被称为**闭包**（closure）——编程中最强大的概念之一。

6.3 可视化代码执行

为了更直观地理解函数和闭包的运行过程，可以使用以下交互式可视化工具：

pythontutor.com

将代码粘贴进去，逐行执行，就能看到 Python 是如何计算表达式、绑定变量以及管理函数调用的。

7 本讲小结

- **基本表达式**——数字、算术运算和布尔运算
- **变量**——用赋值运算符将名称与值绑定
- **函数**——用 `def` 和 `return` 定义可复用的计算单元
- **函数即值**——可以将函数绑定到变量名
- **条件控制**——用 `if`、`elif` 和 `else` 控制程序流程
- **`print` 与 `return`** ——理解副作用与返回值的区别
- **高阶函数**——接收或返回函数的函数；闭包